Is Your AI Truly Yours? Leveraging Blockchain for Copyrights, Provenance, and Lineage

Qin Wang, *Member, IEEE*, Guangsheng Yu, *Member, IEEE*, Yilin Sai, *Student Member, IEEE*, H.M.N. Dilum Bandara, *Member, IEEE*, and Shiping Chen, *Senior Member, IEEE*

***Abstract*—As Artificial Intelligence (AI) integrates into diverse areas, particularly in content generation, ensuring rightful ownership and ethical use becomes paramount, AI service providers are expected to prioritize responsibly sourcing training data and obtaining licenses from data owners. However, existing studies primarily center on safeguarding static copyrights, which simply treat metadata/datasets as non-fungible items with transferable/trading capabilities, neglecting the dynamic nature of training procedures that can shape an ongoing trajectory. In this paper, we present IBIS, a blockchain-based framework tailored for AI model training workflows. Our design can dynamically manage copyright compliance and data provenance in decentralized AI model training processes, ensuring that intellectual property rights are respected throughout iterative model enhancements and licensing updates. Technically, IBIS integrates on-chain registries for datasets, licenses and models, alongside off-chain signing services to facilitate collaboration among multiple participants. Further, IBIS provides APIs designed for seamless integration with existing contract management software, minimizing disruptions to established model training processes. We implement IBIS using Daml on the Canton blockchain. Evaluation results showcase the feasibility and scalability of IBIS across varying numbers of users, datasets, models, and licenses.**

***Index Terms*—AI, blockchain, license, provenance, trust.**

I. Introduction

ARTIFICIAL Intelligence (AI) technologies [1], [2] have increasingly permeated various aspects of daily life, spanning from information retrieval [3], [4] to content generation [5], [6]. Concurrently, AI service providers have made strides in commercializing their services. Nevertheless, as AI models rely on extensive datasets aggregated from diverse sources for training [7], [8], apprehensions have emerged regarding the potential infringement of copyrights [9], [10], [11] during the data acquirement and model training process. To uphold responsible and ethical AI practices [12], [13], comply with regulations, and reduce legal liabilities, AI service providers must actively collaborate with data owners, including content creators and media industry stakeholders. Establishing licensing agreements [14], [15] and obtaining consent before utilizing data for AI model training is a key element of this collaboration [16]. Hence, there is a growing need for new frameworks addressing data provenance, lineage, and copyright compliance in the AI industry, tailored to its distinct needs and workflows.

However, addressing the concerns of AI data provenance and copyright compliance can be a nontrivial task, particularly when the entire training process occurs locally or within a black-box cloud service [17], limiting transparency for users. To bridge this gap, we harness the properties of blockchain technology, which offers a tamper-proof and trustworthy environment [18] to establish authenticity, provenance, and lineage [19], [20]. Owing to its inherent characteristics of immutability and transparency, blockchain has garnered widespread recognition as a suitable technology for achieving regulatory compliance [22], [23]. For instance, data recorded on the blockchain is digitally signed and inherently tamper-proof, thereby constituting an authentic and persistent record that accurately reflects an event(s) at a specific point in time. This makes blockchain a fitting candidate to address concerns related to data provenance and copyright compliance within the AI industry.

We have identified a series of functional challenges for such a blockchain-based compliance framework [27], [28], [29]: (i) The framework needs to be designed to seamlessly integrate with the existing workflow of AI model training [30], [31], [32]. (ii) The framework should support continuous model retraining and fine-tuning with new datasets [33], allowing for the generation of updated models while maintaining data provenance and lineage. (iii) The framework should support mechanisms for license expiration and renewal, accommodating diverse business models employed by data owners. (iv) The ownership of datasets and models [34], [35], along with all training actions, should be accompanied by evidence to clarify their licensing scope and ensure accountability for any subsequent actions. (v) The framework should facilitate communication between AI service providers and data owners [19], [36], enabling efficient attainment and documentation of licensing agreements. (vi) The framework should ensure the effective management and commercial sensitivity of licenses, safeguarding them against unauthorized access by third parties [37].

In this paper, we design, implement, and evaluate IBIS, a blockchain-based framework for data and model copyright management, provenance, and lineage in AI model training processes. IBIS empowers model owners to establish the provenance and lineage of their AI models and training datasets throughout retraining and fine-tuning processes, efficiently obtaining copyright licenses from the relevant copyright holders, and securely recording and renewing bilaterally signed copyright licenses as

Received 31 January 2025; revised 15 June 2025; accepted 24 June 2025. Date of publication 3 July 2025; date of current version 9 October 2025. *(Corresponding author: Guangsheng Yu.)*

Qin Wang, Yilin Sai, H.M.N. Dilum Bandara, and Shiping Chen are with the CSIRO Data61, Eveleigh, NSW 2015, Australia, and also with the University of New South Wales (UNSW), Sydney, NSW 2033, Australia (e-mail: qin.wang@data61.csiro.au; wjsylhh95@gmail.com; dilum.bandara@data61.csiro.au; shiping.chen@data61.csiro.au).

Guangsheng Yu is with the University of Technology Sydney, Ultimo, NSW 2007, Australia (e-mail: guangsheng.yu@uts.edu.au).

Digital Object Identifier 10.1109/TSC.2025.3586091

1939-1374 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on April 15,2026 at 09:46:28 UTC from IEEE Xplore. Restrictions apply.

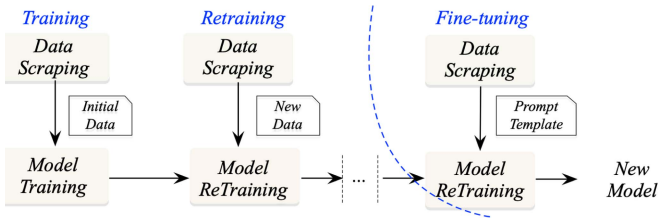


Fig. 1. Typical AI model training process.

evidence of legal compliance. Our detailed contributions are as follows:

- *We propose a blockchain-integrated framework,* IBIS*, to track data and model copyright management, provenance, and lineage:* IBIS exhibits the following characteristics:
  - Seamless integration (addressing c-i);
  - Adaptability (addressing c-ii and iii);
  - Traceable registry (addressing c-iv);
  - Blockchain-based multi-party signing (addressing c-v);
  - Controllability (addressing c-vi);
- *We implement a fully-functional prototype based on the Daml smart contract language [38] and Canton blockchain protocol [39].* We adopted Daml and Canton's renowned privacy-preserving capabilities and modular design to implement a secure and commercial-sensitivity-preserving framework with six modules dedicated to license registration, management, and updating.
- *We conduct a series of performance evaluations of* IBIS*, especially its performance under a parameterized real-world scenario.* Evaluation results show that a model owner can retrieve a model's datasets and its licenses in approximately 1.5 and 3 seconds, respectively. This is irrespective of the number of model owners, datasets, and licenses hosted within the framework. Additionally, retrieving authorized models for a license takes approximately 1.5 seconds, regardless of the number of training datasets per model, model owners, and licenses within the framework. These results demonstrate scalability under varying numbers of users, datasets, models, and licenses.

The rest of the paper is organized as follows: Section II provides background and related work. Section III gives the system architecture and our design. The construction details of our framework, including data models and functional operations, are presented in Section IV. Sections V and VI present our implementation with performance evaluations. Section VII offers conclusions and suggests avenues for future research.

II. Technical Warm-Ups

A. Background

AI model training: In general, the training process for AI models is continuous and iterative, containing *training*, *retraining*, and *fine-tuning* [40]. As seen in Fig. 1, training begins with data collection, where initial training datasets are gathered through *data scraping*. These datasets are then fed into the *model training* step, where a preliminary model is trained. To ensure the model remains effective and up-to-date, it undergoes periodic retraining with newly collected data, allowing it to adapt to new information. Additionally, a model may undergo a *fine-tuning* phase, where it is slightly retrained to meet specific domain requirements, enhancing its accuracy and relevance for targeted applications.

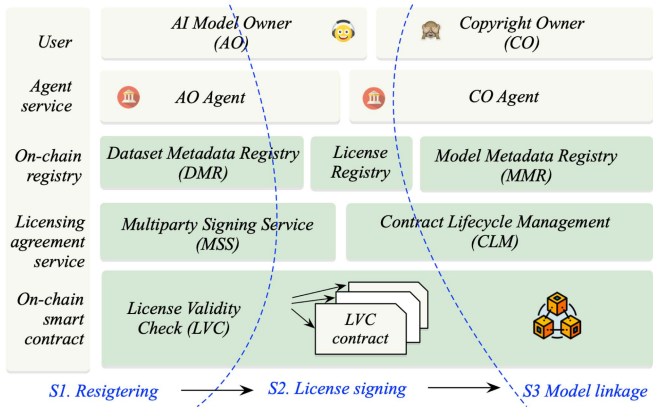


Fig. 2. Architectural design.

Copyrights: Copyright grants creators exclusive rights to their original expressions, such as literary, artistic, and musical works. This legal framework safeguards creators' rights, allowing them to control how their work is used, reproduced, and distributed.

B. Related Work

Protecting copyright/data in AI: Data and copyright protection in AI services is a long-standing topic. Existing methods can be classified in several aspects [50]: Data-modifying approaches involve modifying or sanitizing user data to unlink them from specific individuals (e.g., k-anonymity [51], differential privacy [52], and watermarking [53]). This minimizes the risk of reidentification by removing or concealing Personally Identifiable Information (PII). Data-encrypting approaches encrypt user data to ensure integrity and confidentiality during data sharing, leveraging techniques such as homomorphic encryption [54] and secure Multi-Party Computation (MPC) [55]. Data-minimizing approaches aim to boost efficiency by reducing the volume of personal data needed [56], often observed in general model training where PII data are not required during training and minimally during inference. Data-confining approaches involve AI methods that operate without sharing PII data beyond user boundaries [57], ensuring data integrity and confidentiality while enabling effective personalization through local access to personal data.

Blockchain-empowered copyright management: Liang et al. [58] employed smart contracts to establish a homomorphic encryption mechanism aimed at safeguarding circuit copyrights. Liu et al. [59] employed a blockchain-based fraud-proof protocol to secure ownership rights over AIGC (artificial intelligence-generated content). Numerous similar solutions are outlined in studies such as [43], [44], [45]. It is worth noting that most existing blockchain-based studies treat each copyright merely as a form of non-fungible online property, akin to an NFT. However, this approach restricts its practical utility in real-world scenarios that require varied operations like registration, renewal, and termination – features that our framework offers in contrast.

III. Proposed Design: IBIS

A. Design Overview

Roles: We distinguish between two pivotal roles: *AI Model Owners* (AOs) and *Copyright Owners* (COs). AOs act as representatives of the creators or uploaders of AI models,

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on April 15,2026 at 09:46:28 UTC from IEEE Xplore. Restrictions apply.

who construct, train, maintain, and commercialize the AI models. In our framework, their responsibilities include the categorization of data, acquirement of licenses, and registration of dataset/model metadata onto the blockchain. COs are the rightful copyright holders of training data with the authority to license their data, encompassing content creators and media companies among others. Their involvement extends to the drafting and bilateral signing of license agreements, ensuring regulatory compliance, and the protection of intellectual property rights. Likewise, a foundational model owner is also a CO from the point of view of an extended AO. In this scenario, datasets utilized in developing the foundational model are licensed by a separate set of COs, encompassing the licensing of derivative works. Distinguishing between these two CO roles is not imperative within IBIS, as it can effectively keep track of complex data and model relationships (see Section IV-B).

Architecture: We envision an ecosystem that empowers both AOs and COs to harness the network effect of a unified platform to train and use numerous AI models and datasets. For example, CO could license the same dataset to multiple AOs and reap the benefits of a pay-as-you-go model for dataset usage or derived work within the same platform. Therefore, our proposed framework, IBIS (cf. Fig. 2), is designed to integrate a blockchain network hosted by a subset of AOs and COs. While established and commercially significant AOs and COs may operate blockchain nodes, others only require the capability to connect to one of them via an agent.

At its core, the blockchain network serves as the backbone, facilitating secure and transparent interactions between AOs and COs. The system is architected to abstract the complexities of blockchain interaction through an agent service, offering user interface, authentication, and request buffering. The agent service acts as a bridge, connecting the AOs and COs with the blockchain, thereby enabling efficient metadata registering and licensing processes. Additionally, the system architecture includes dedicated components for handling dataset metadata registration, bilateral license signing, and model metadata management, each playing a crucial role in the overall workflow of AI model training and copyright handling.

B. Key Modules

IBIS has the following six main modules (green in Fig. 2):

- *Dataset Metadata Registry* (DMR) maintains an on-chain metadata record for each dataset scraped by AOs. These records include details such as the dataset's CO and its source URL.
- *License Registry* records copyright licenses that are bilaterally signed by the corresponding CO and AO, serving as evidence of data use agreements. When a dataset is licensed, a two-way linkage is established between the license record and the DMR record of the dataset. A single license may cover multiple datasets.
- *Model Metadata Registry* (MMR) stores the metadata of a model once it has been trained. This metadata includes the model's identifier, as well as the identifiers of training datasets and source models. It maintains a persistent record of the datasets and source models utilized in models' training, thereby establishing data provenance.
- *Multi-party Signing Services* (MSS) orchestrate communication between AOs and COs. It handles tasks such as sending license request emails to COs and returning license drafts to AOs. Most importantly, leveraging the identity management and digital signature capabilities of the blockchain, MSS ensures secure multi-party signing processes for establishing copyright licenses.
- *Contract Lifecycle Management* (CLM) provides a unified API to interface with various external CLM software solutions that manage licenses. This approach ensures compatibility with a range of CLM software solutions, minimizing disruption to COs' existing workflows.
- *License Validity Check* (LVC) employs smart contracts to verify the validity of a license based on a set of environment variables, including the current date, AO's operating location, and any other variables that could potentially contravene terms and conditions stipulated in the license. Our framework allows the creation of custom LVC smart contracts targeting different licence types.

Notably, both MSS and CLM are implemented as off-chain services. They are essential for facilitating interactions that precede and follow blockchain transactions, but they do not operate directly on the blockchain.

C. Stages

For the initial model training, the workflow of our framework can be segmented into the following three stages:

S1. Dataset registering and license check: This involves dataset categorization, metadata registration, and license checks via smart contracts to ensure copyright compliance.

Specifically, the workflow begins with dataset categorization, where datasets are organized into specific categories based on their content, source, and usage. This categorization facilitates efficient retrieval and management of datasets throughout the AI model development process. Following categorization, metadata registration takes place via DMR, recording crucial details such as data descriptions, authorship information, and usage rights within a structured format. This step ensures that comprehensive information about the datasets is readily accessible and referenced during their lifecycle. Finally, license checks are conducted via LVC smart contracts, utilizing automated processes to verify the authenticity and compliance of licenses associated with the datasets. Smart contracts ensure that AI model training and usage adhere to copyright agreements, providing a streamlined and compliant workflow for managing datasets and their associated licenses.

S2. License drafting and bilateral signing: In case of failed license checks, this stage involves drafting and bilateral signing of licenses, facilitated by the MSS and CLM.

Upon identifying any missing, expired, or reworked licenses during stage S1, the process swiftly progresses to crafting bespoke license agreements tailored to the unique datasets and their intended applications. Leveraging recent advances in MSS technology, stakeholders embark on bilateral negotiations to refine the terms of these agreements, culminating in their formal agreement/contract through digital signatures. Finalization of agreements occurs only upon the attainment of signatures from both parties. Subsequently, CLM solutions seamlessly interface with current systems, streamlining contract management duties including drafting, approval workflows, and compliance oversight.

S3. Model registering and copyright owner notification: In this stage, post-training models are recorded in on-chain MMR, creating a bidirectional linkage between models and training datasets. Notifications may be dispatched to COs as stipulated by the licensing agreement.

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on April 15,2026 at 09:46:28 UTC from IEEE Xplore. Restrictions apply.

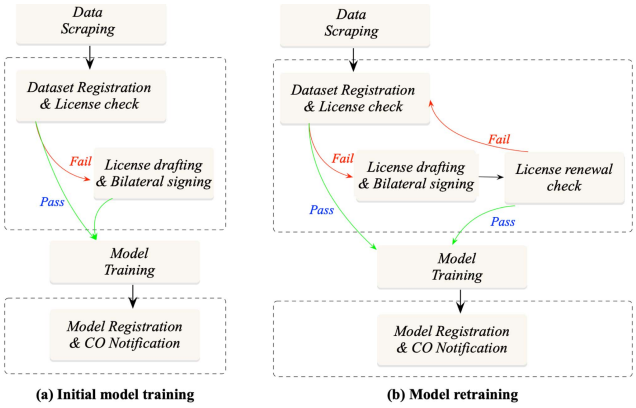


Fig. 3. Integration with existing AI workflow.

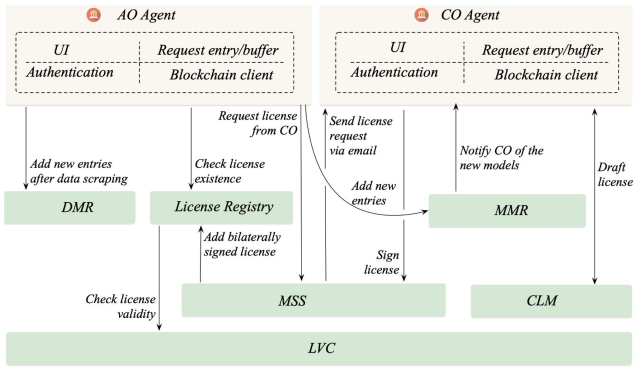


Fig. 4. Functional design across each module.

D. Details of Each Stage

The flowchart in Fig. 3(a) illustrats how IBIS is integrated into existing AI model training. Next, we discuss the main stages in detail, and Fig. 4 depicts interactions across IBIS modules.

1) Dataset Metadata Registering and License Check: During or after the initial data scraping process, AOs categorize the data into one or more datasets and register the metadata of each dataset on the blockchain. The on-chain DMR maintains a metadata record for each dataset, containing details such as its CO[1] and source URL (refer to Section IV-A for detailed information on data model specifics). Notably, IBIS operates under the assumption that COs are uniquely identifiable, and the AO organizes the data in such a manner that each dataset is associated with only one copyright owner. A corner case arises when a dataset originates from the public domain and therefore lacks a distinct owner. In such instances, we stipulate the use of a designated copyright owner identifier, "public-domain", to account for public domain data.

Datasets registered in DMR are not automatically eligible to become training data. To adhere to copyright laws, each dataset must undergo a vetting step, involving a check for the existence and validity of its license. During this step, AO extracts attributes of a dataset and queries the license registry on the blockchain for a corresponding license. Section IV-B describes the license registry search mechanism. Once a license is found, its validity is checked using the LVC smart contract. Only a dataset that passes the license check is deemed eligible for model training. Regarding the corner case of public domain data, the license registry is preloaded with a public domain license that always passes the LVC. Here, the mechanism of an LVC can vary depending on the type of license. For instance, certain licenses may impose geographic restrictions, while others may have time or number of use limits. Consequently, our framework facilitates the creation of different LVC smart contracts to accommodate such diverse and complex conditions. We define a generic LVC interface contract to dynamically determine which specific LVC contract to utilize based on the license being evaluated.

[1]The method for acquiring copyright owner's information during data scraping is out of the scope of this paper.

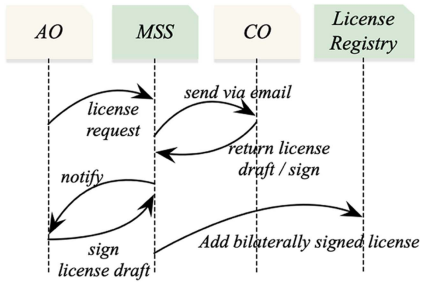


Fig. 5. MSS workflow.

After the dataset successfully passes the license validity check, the licenseId attribute in its metadata will be updated to reference the valid license. Additionally, the dataset's identifier will be added to the license's datasetList attribute. This establishes a bidirectional linkage between a dataset and its corresponding license.

2) License Agreement Drafting and Bilateral Signing: If the license existence or validity checks fail, AO must initiate a license agreement drafting and signing stage to obtain a license from the CO. This stage commences with AO sending a license request to the corresponding CO. This request is routed through MSS, which then generates an email containing the request details and along with a link for CO to take necessary actions. One of the actions involves drafting a license agreement based on the request. The CO executes this drafting action by invoking the API via CLM. Connectors that interface with various external CLM software solutions implement this API. This design is predicated on the understanding that COs often rely on proprietary software to draft their licensing agreements and manage data subscriptions. Thus, we leverage the CLM's API and connectors to ensure compatibility with a range of existing CLM software solutions, minimizing disruption to the copyright owners' existing workflow.

Once the CO drafts the license agreement using the CLM software, the agreement is transmitted back to the framework via the CLM connector and API. Subsequently, CO and AO engage with the MSS to generate a bilaterally signed license agreement (cf. Fig. 5). The signed license agreement is then recorded in the license registry. The dataset's DMR record is also updated to reference the newly acquired license, thereby concluding the license agreement drafting and signing stage. Here, MSS utilizes an email list comprising the email addresses of AOs and COs. Email addresses can be added during user signup or input by the counterparty.

3) Model Registration and CO Notification: The previous two steps empower the AO to accumulate a pool of licensed datasets that can be legitimately employed for AI model training. To establish data and model provenance, it is imperative to

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on April 15,2026 at 09:46:28 UTC from IEEE Xplore. Restrictions apply.

maintain a reliable record of the datasets and hyper-parameters utilized in each model's training. This is accomplished through the creation of MMR on the blockchain. Following the training of a model, its metadata, comprising its identifier, hyper-parameters, and the identifiers of the training datasets, are recorded on the MMR.

Furthermore, for each training dataset, its DMR record is updated to include the identifier of the new model. This establishes a bidirectional linkage between a model and its training dataset. Finally, depending on the terms outlined in the licensing agreement, the framework may dispatch notifications to the respective copyright owners, informing them of the new model trained using their data.

E. AI Model Retraining and Fine-Tuning

The system architecture outlined above not only supports initial model training but also facilitates model retraining and fine-tuning, wherein newly collected data are integrated into the model. As new data are scraped and collected, DMR continues to expand, while new licenses are acquired and stored in the license registry, ensuring the legitimacy of new datasets. Consequently, during the AI model retraining and fine-tuning, the initial two stages remain unchanged.

However, in retraining and fine-tuning scenarios, it is possible that the original model may need a license renewal at the time of retraining or fine-tuning, as one or more licenses of its training datasets might have become no longer valid (e.g., expired). Therefore, before retraining or fine-tuning the model, an additional stage is necessary to verify data eligibility by determining if the model requires a license renewal. Fig. 3(b) depicts the flowchart of actions during model retraining or fine-tuning. Section IV-E presents the detailed mechanism used to conduct the license renewal check.

Moreover, compared to the stages in the initial training process, a divergence arises in the final stage concerning the establishment of data provenance. The new model is a culmination of the original model merged with new datasets. Hence, when recording a new entry in MMR for the retrained/fine-tuned model, in addition to recording the identifiers of the training datasets, the AO must also record the identifier of the original model. This facilitates data provenance and model lineage throughout the iterative model retraining or fine-tuning process. Meanwhile, the metadata of the original model must be updated to include a reference to the new model, thus establishing a bidirectional linkage between the new model and its source model. Complete information on model metadata specifics can be found in Section IV-A.

IV. IBIS With Detailed Workflow

A. Data Models

The main attributes that IBIS framework supports can be broadly grouped as follows (see Table I):

- *Dataset attributes:* A dataset is uniquely identified by a datasetId and sourced from a specific URL. It includes information on the copyright owner, associated license, and models trained on it. Additionally, the dataset's ownership and creator are tracked through the CO's copyrightOwnerId.
- *License attributes:* Each license has a distinct licenseId and encompasses a defined scope, typically a URL/URI. It includes details like copyright ownership, digital signatures of owners, and validity timestamps. The license type identifier typeId aids in determining the applicable LVC smart contract for license validation. Moreover, it lists the datasets covered under the terms of the license.
- *Model attributes:* AI models are identified by a unique modelId and associated with owners. They utilize datasets for training, which are listed within the model's attributes. Retrained models reference a source sourceModelId, and any subsequent models derived from it are listed as child models in childModelList. This structure establishes the lineage of models and facilitates the tracking of relationships between models and data within AI services.

TABLE I
Dataset, License, and Model Attributes

	Atttributes	Descriptions
Dataset	datasetId	Unique identifier of the dataset.
	sourceUrl	URL from which the dataset was scraped.
	copyrightOwnerId	Unique identifier of the copyright owner.
	licenseId	Unique identifier of the copyright license.
	modelList	List of unique identifiers of the models trained on this dataset.
	modelOwnerId	Unique identifier of AO who scrapes and adds this dataset.
License	licenseId	Unique identifier of the dataset.
	scope	URL to dataset. Datasets pointed by this URL fall within the scope of the license.
	copyrightOwnerId	Unique identifier of the copyright owner.
	copyrightOwnerSignature	Digital signature of the copyright owner.
	modelOwnerId	Unique identifier of the model owner.
	modelOwnerSignature	Digital signature of the model owner.
	validFrom	Timestamp indicating when the license takes effect.
	typeId	Unique identifier of the license type. Used to determine the smart contract to check the license validity.
	datasetList	List of identifiers of the datasets covered by this license.
Model	modelId	Unique identifier of the AI model.
	modelOwnerId	Unique identifier of the model owner.
	datasetList	List of unique identifiers of training datasets.
	sourceModelId	Unique identifier of the source model that has been retrained.
	childModelList	List of identifiers of the models trained based on this model.

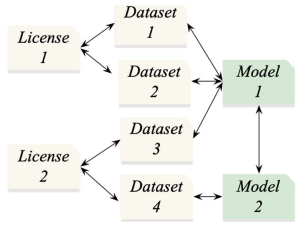


Fig. 6. Model, dataset, and license relationships.

We highlight two aspects. First, the data model is extensible, enabling AOs and COs to incorporate additional custom attributes as needed. For example, a storage URL can be included in dataset metadata to indicate where AO stores the dataset. A license can include custom attributes such as expiration date, exclusivity, and other terms and conditions. Second, a web path is employed to delineate the scope of what is being licensed, considering that the majority of AI models are trained using online data.

A running example: Fig. 6 illustrates the logical interrelation among the three data models, within an example scenario where Model-1 is initially trained using three datasets, and subsequently retrained with a fourth dataset to yield Model-2. The two models are linked through Model-2's sourceModelId attribute and Model-1's childModelList. A dataset and a model are linked through the model's datasetList and the dataset's modelList. A license and a dataset are linked through the license's datasetList and the dataset's licenseId.

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on April 15,2026 at 09:46:28 UTC from IEEE Xplore. Restrictions apply.

Integrity binding with off-chain data: While the actual datasets and models are stored off-chain, IBIS ensures integrity through the use of cryptographic hashes. Each dataset and model is associated with a content-addressable identifier, typically computed using a collision-resistant hash function (e.g., SHA-256). These hashes are recorded on-chain as part of the metadata entries in the DMR and MMR smart contracts. As a result, any tampering with off-chain content can be detected by recomputing and comparing the hash values.

Optionally, the system may integrate with decentralized storage platforms (e.g., IPFS), where the content hash also serves as the retrieval address. This dual use of hashes enhances verifiability and offers a robust mechanism for ensuring data provenance and integrity even in off-chain storage.

B. Obtaining Licenses

Obtain dataset licenses: The getDatasetLicense operation retrieves copyright license of a given a dataset identifier datasetId. It initially searches the DMR hash map using the dataset datasetId as the key, which incurs a time complexity of $O(1)$. Depending on whether the returned DMR record contains a lisenceId, this operation either retrieves the license by licenseId or searches for a relevant license in the license registry as follows:

- *Retrieve the license with* licenseId: This operation queries the license registry hash map using the licenseId as the key. As the resulting time complexity is $O(1)$, the overall time complexity remains the same.
- *Search for a relevant license:* This operation extracts the dataset's copyrightOwnerId and performs a search on the license registry using copyrightOwnerId, which involves a complexity of $O(1)$. This search may yield a list of licenses with the same copyrightOwnerId (albeit with different scopes). Finally, a scan is conducted on the list of licenses to filter out the licenses with irrelevant scopes. Our framework operates on the premise that license scopes do not intersect, ensuring each dataset corresponds to at most one license. As CO is unlikely to have many licenses with the same AO for practical reasons, one can assume the size of this list to be small.

Obtain model licenses: The getModelLicenses operation retrieves the licenses for its training datasets, given a modelId. It first identifies the datasets linked to the model and its upstream sources, then follows graph edges—model-to-dataset, dataset-to-license, and model-to-model (cf. Fig. 6)—to collect the associated licenses. For a given model, the operation traverses its T linked datasets, retrieves their licenses, and deduplicates them if needed. The time complexity is $O(T)$, as it processes one edge per dataset. When applied to all M models, the operation has a total complexity of $O(|M||T|)$, where T is the number of datasets per model. As $T \ll D$ (total datasets), the traversal remains focused and efficient, scaling linearly with the number of models and their training datasets.

C. Validating Licenses

Check license validity: Given license data and environment variables as transaction inputs, the checkLicenseValidity operation verifies the validity of the license. Environment variables include the current date, the operating locations of AOs, and other variables that could potentially contravene the terms and conditions stipulated in the license agreement.

First, we need to locate the corresponding LVC smart contract for validating the license. This can be accomplished using another hash map where the license type typeID serves as the key and LVC's address as the value. Consequently, this lookup operation can be performed in constant time, i.e., $O(1)$. Next, we need to invoke the identified LVC contract to determine the license validity. The time complexity of the LVC contract is directly proportional to the number of environment variables that need validation. We abstract this time complexity as $O(|K|)$, where $\mathbf{K}$ is the set of environment variables to validate. The overall time complexity is $O(1) + O(|K|) = O(|K|)$.

Here, we give an example (Listing 1). This LVC template is triggered by the checkLicenseValidity operation and verifies that the license is currently active based on temporal conditions. The modular design of LVCs allows for extensible contract templates supporting various types of constraints (e.g., geographic, exclusivity, usage count), which can be selected based on the license's typeId.

```
template TimeBoundLVC with
    licenseId: Text
    validFrom: Time
    validUntil: Optional Time
    currentTime: Time
  where
    signatory licenseVerifier

    choice CheckValidity : Bool
      controller licenseVerifier
      do
        let isValidStart = currentTime >= validFrom
        let isValidEnd = case validUntil of
                           Some end -> currentTime
     <= end
                           None -> True
        return (isValidStart && isValidEnd)
```

Listing 1: Example LVC smart contract template for validating a time-bound license.

D. Obtaining Data and Models Via Licenses

Obtain licensed datasets: The getLicensedDatasets operation retrieves the list of dataset identifiers datasetIds each with a valid license. It entails executing getDatasetLicense and checkLicenseValidity operations for each dataset. Given $\mathbf{D}$ datasets, the overall time complexity is $O(|D| \times \{O(1) + O(K)\} = O(|D||K|)$.

Obtain authorized datasets by license: Given a license identifier licenseId, the getDatasetsByLicense operation retrieves datasets covered by the license. This operation performs a search of the DMR using the licenseId, resulting in a time complexity of $O(1)$.

Obtain authorized models by license: Given a license identifier licenseId, the getModelsByLicense operation retrieves the metadata of the models covered by the license. This operation begins by executing getDatasetsByLicense, which identifies all datasets associated with the given license. For each of these datasets, the operation performs a graph traversal to retrieve the models directly trained on them and any child models indirectly connected through shared datasets. The overall time complexity consists of two components: $O(|D|)$ for identifying the datasets linked to the license and $O(|M||T|)$ for traversing the graph to

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on April 15,2026 at 09:46:28 UTC from IEEE Xplore. Restrictions apply.

retrieve the connected models and their relationships. Consequently, the total complexity is $O(D + |M||T|)$, where D is the total number of datasets, M is the total number of models, and T is the average number of datasets per model.

Obtain model datasets: Given a model identifier modelId, the getModelDatasets operation retrieves the identifiers of its training datasets. During the first traversal or initialization, this operation iterates through the provided model and its upstream source models, retrieving the associated datasets. If the model has a set of $\mathbf{M}$ upstream models, each linked to $\mathbf{T}$ datasets, the time complexity is $O(|M||T|)$ for the initialization phase.

E. Renewing License

License validity check is an ongoing task because a valid license may become valid under certain circumstances (e.g., revoked or expired), necessitating AOs and COs to take appropriate actions to ensure continuous compliance with copyright laws. Following delineates how the framework facilitates license renewal checks and renewals.

License renewal check: The framework supports three types of license renewal checks (LRCs): license-driven, dataset-driven, and model-driven.

In license-driven LRC, AOs or COs conduct a periodic scan of the license registry, performing checkLicenseValidity on each license. If a license fails the validity check, an AO can execute the getModelsByLicense operation to gather the identifiers of datasets and models that depend on the valid license. These can be flagged and recorded in a list of disallowed resources to prevent their use in future training of new models or retraining. The specifics of how the list of disallowed resources is stored and managed fall beyond the scope of this paper.

Dataset-driven and model-driven LRC can be conducted on-demand before training a new model. In dataset-driven LRC, an AO can execute getDatasetLicense operation followed by the checkLicenseValidity operation for each training dataset to identify any dataset needing a license renewal. In contrast, in model-driven LRC, an AO can execute getModelLicenses operation followed by checkLicenseValidity operation for each license of the model, determining whether the model needs a license renewal.

License renewal: A license renewal involves adding a new bilaterally signed license to the license registry, rather than updating existing records. This enables AOs and COs to access all historical licenses to prove regulatory compliance and avoid any disputes. After a new license has been added, an AO can execute the getModelsByLicense operation to gather the identifiers of datasets and models that depend on the renewed license. Then the DMR records of datasets are updated to reference the new license. As the list of dependent datasets and models can now be considered eligible for training, their identifiers are also removed from the list of disallowed resources.

F. More Operational Features

Operation atomicity: It is observed that several stages (i.e., *S.1*, *S.3*, and License Renewal) involve the update of multiple records. Apart from ensuring integrity and immutability, another advantage of maintaining DMR, license registry, and MMR on-chain is that such multiple updates are guaranteed to be atomic. This is because smart contracts can ensure atomicity where the actions included in one transaction either all take effect or none of them take effect. Therefore, care needs to be taken during implementation to ensure that all updates within a stage should be included in the same transaction.

TABLE II
RESOURCES AND AUTHORISATIONS

Resource	Authorised Actor	Access Scope
DMR	AOs	Datasets added by the actor
License Registry	AOs/COs	Licenses signed by the actor
MMR	AOs	Models owned by the actor
LVC API	AOs/COs	Licenses signed by the actor
CLM API	COs	Licenses drafted by the actor

License linkage: In IBIS, each dataset is assigned a license at the time of registration through the licenseId attribute. This linkage is immutable and ensures that all subsequent uses of the dataset are bound by the same licensing terms. The license itself enumerates the covered datasets via its datasetList attribute. This bidirectional mapping supports consistent enforcement of usage policies and enables automatic validation of licensing conditions during model training and retraining. The use of type-specific LVC contracts (see Section IV-B) ensures that the correct license validation logic is applied in each case.

Dataset reuse: When a dataset is reused across multiple models, original licensing terms remain applicable. The reuse is authorized as long as the license associated with the dataset remains valid. During model creation or retraining, the system verifies license validity through checkLicenseValidity and ensures that all datasets listed in the model's datasetList are covered by licenses returned by getModelLicenses. Since licenses may impose usage constraints (e.g., valid period, geographic restrictions), this step guarantees ongoing legal compliance for every instance of dataset reuse.

Traceability: To support auditing and provenance, IBIS explicitly records the lineage of models and datasets. Each dataset maintains a modelList tracking all models trained on it, while each model retains a datasetList identifying its training inputs. For retrained models, the sourceModelId and childModelList attributes form a directed acyclic graph (DAG), capturing the complete history of derivations. These structural links are embedded in the smart contract state and are queryable via the getModelDatasets and getModelsByLicense operations. As a result, IBIS provides fine-grained, tamper-proof traceability across both horizontal (multi-model) and vertical (model-chain) reuse scenarios.

V. IMPLEMENTATION

A. Overview

Prototype stack: We deploy Canton [39] network (v2.8.3) with three controlled nodes on AWS t2.xlarge instances (configuration by 4vCPU, 16GB RAM, Ubuntu22.04). Each node runs a *Participant+Synchroniser* pair backed by Docker-ised PostgreSQL15.

Component map: Fig. 7 links the on-chain contracts (DMR, MMR, LICENSE) to off-chain services (MSS, CLM); Table II lists party-centric access scopes enforced by Canton.

Code footprint: Core Daml templates ≈ 380 LOC, HTTP/gRPC glue ≈ 210 LOC, evaluation harness ≈ 150 LOC.

Reproducibility: We wrap up experiments and a single command line (i.e., $ docker − composeup) can instantiate the full

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on April 15,2026 at 09:46:28 UTC from IEEE Xplore. Restrictions apply.

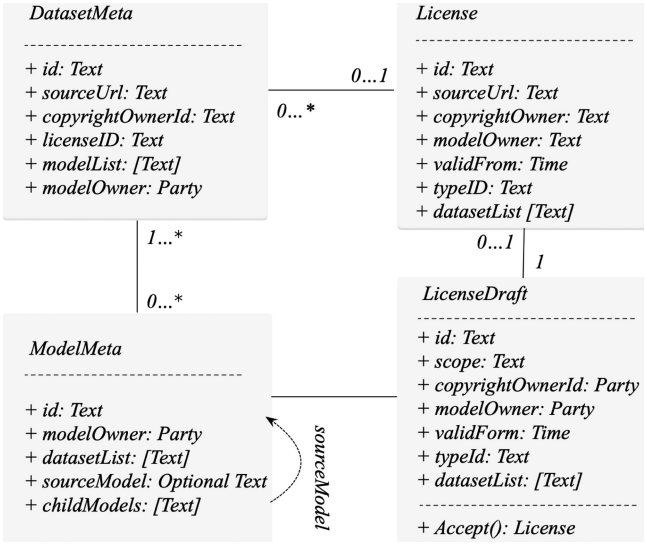


Fig. 7. The class diagram of IBIS implementation.

network and re-run Experiments 1–4; scripts are open-sourced at https://github.com/yilin-sai/ai-copyright-framework.

B. Blockchain Platform

We implement IBIS in **Daml** (Digital Asset Modelling Language) atop the **Canton** ledger protocol.

Contract-as-data model: Daml templates codify both schema and behaviour. We can map the Dataset/Model/License records of Table I to verifiable contracts with one-to-one clarity.

Ledger-level privacy: Canton extends Daml's role system (*signatory*, *controller*, *observer*, *choice-observer*) so each node synchronises *only* those contracts it is authorised to see; commercial terms inside a LICENSE stay invisible to third parties even on-ledger.

- *observer:* A party that is guaranteed to see actions that create and archive the contract.
- *controller:* A party that can exercise a particular choice (i.e., invoke a function) on the contract.
- *choice observer:* A party that is guaranteed to see a particular choice being exercised on the contract.

Why not Fabric / generic EVM? Hyperledger Fabric chaincode can enforce ACLs but still replicates the entire channel ledger to all members [60]. Public-chain EVM contracts disclose everything. Canton's party-centric sync therefore offers the best trade-off between immutability and confidentiality for multi-tenant AI marketplaces.

Therefore, by adopting Daml and Canton, we facilitate the integration of many AOs and COs onto the same IBIS platform, while ensuring that commercially sensitive license agreements between an AO and COs, dataset metadata, and model metadata remain concealed from other parties, even at the ledger level. Table II lists the resources in IBIS along with their authorizations.

C. Smart-Contract Implementation

Listing 2 illustrates the license smart contract template, constructed based on the data schema in Table I. The copyrightOwner and modelOwner are designated as signatories of the template (Line 10). This reflects the bilateral nature of the license agreement. Consequently, authorization from both parties is required to create a license contract. Subsequently, once the contract is created, both parties can access it, while no other party has access to this contract either on-chain or on-ledger. Hence, this implementation closely aligns with the access control requirements specified in Table II. In addition, the identifier attribute of each license serves as the primary key (specified by Line 12-13), which can facilitate efficient queries during the graph traversal.

```
1  template License with
2      licenseId: Text
3      scope: Text
4      copyrightOwner: Party
5      modelOwner: Party
6      validFrom: Time
7      typeId: Text
8      datasetList: [Text]
9    where
10     signatory copyrightOwner, modelOwner
11
12     key (modelOwner, licenseId) : (Party, Text)
13     maintainer key._1
```

Listing 2: Daml smart contract template for license.

Similarly, dataset and model metadata are represented in smart contract templates according to their corresponding data models, except that only the modelOwner is designated as the signatory of the DatasetMeta and ModelMeta contracts. This setup enforces two access control rules:

- Only modelOwner has the authority to create and access these contracts.
- modelOwner is restricted to accessing DatasetMeta or ModelMeta contracts created by themselves.

Full implementation details are open sourced. Fig. 7 illustrates the class diagram of IBIS design.

D. Multi-Party Signing

The generation of a bilaterally signed license contract adheres to Daml's *Propose and Accept Pattern.*[2] In this pattern, one party initiates a proposal contract, which the counterparty can either accept or reject. The acceptance (or rejection) is implemented as a contract choice[3], allowing the counterparty to exercise their decision. Upon exercising the accept choice, a result contract is generated, symbolizing the agreement between the two parties.

In our implementation, the proposal contract takes the form of the draft license agreement, and it is depicted in Listing 3. LicenseAgreement shares the same data schema as License (see Listing 2), with the copyrightOwner designated as the signatory. This ensure only the copyrightOwner has the authority to create a contract. Alternatively, modelOwner, as the controller of the Accept choice in Line 16, has the authority to exercise the choice, resulting in the creation of a License contract. This setup ensures that only the modelOwner specified in a LicenseAgreement contract has the authority to exercise the Accept choice of that contract.

[2]https://docs.daml.com/daml/patterns/initaccept.html#the-propose-and-accept-pattern.

[3][Online]. Available: https://docs.daml.com/daml/intro/4_Transformations.html#choices-as-methods

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on April 15,2026 at 09:46:28 UTC from IEEE Xplore. Restrictions apply.

```
template LicenseAgreement with
    id: Text
    scope: Text
    copyrightOwner: Party
    modelOwner: Party
    validFrom: Time
    typeId: Text
    datasetList: [Text]
  where
    signatory copyrightOwner

    key (copyrightOwner, id) : (Party, Text)
    maintainer key._1

    choice Accept: ContractId License
      controller modelOwner
      do create License
          with licenseId; scope; copyrightOwner;
     modelOwner; validFrom; typeId; datasetList
```

Listing 3: Daml smart contract template for license agreement.

E. Operational Footprint

Gas / storage: A complete write–read cycle touches four contracts and stores ≈ 3.5 kB; even with 10^3 datasets/models/licenses the ledger is ≤ 15 MB.

Performance hooks: Primary-key indices and batched Canton commits keep the $P95$ end-to-end latency below 1.2s at 1 000 req/s (§VI) without heavyweight zk-proofs or HE / sMPC.

VI. Evaluation

Our proof-of-concept IBis implementation was deployed on a private Canton blockchain comprising three nodes. All nodes were hosted on the same AWS EC2 t2.xlarge instance with four virtual CPUs and 16GB of RAM. While Daml provides a range of options for data storage, PostgresSQL, running within Docker containers, is chosen as the data storage to persist node data. Our source code of the performance test is available. In the following sections, we present four sets of experiments: (1) baseline evaluation of core search operations, (2) comparative analysis of execution overhead against other blockchain-based designs, (3) scalability under varying load and storage conditions, and (4) robustness against adversarial attacks targeting provenance and signature integrity.

A. Experiment-1: Searching Operations

Our evaluation mainly focuses on three operations involving graph traversals: fetching model licenses using getModelLicenses, model datasets using getModelDatasets, and authorized models using getModelsByLicense. The former two operations pertain to copyright management, while the latter concerns data provenance. To enhance the accuracy of performance testing, for every parameter configuration, the operation of getModelLicenses is executed ten times on ten randomly chosen models, with the average execution time and standard deviation calculated thereafter. Similarly, getModelDatasets undergoes execution on ten randomly selected models. As for getModelsByLicense, the operation is performed on ten randomly chosen licenses, with the resultant average execution time and standard deviation recorded.

Benchmarks and baselines: In the absence of a unified benchmark for evaluating methods in this context, we focus on execution time as the primary evaluation criterion. This metric is reasonable and sufficient given that our method, IBis, is implemented on a blockchain and emphasizes efficient searching when providing copyright protection for models and datasets. Moreover, the focus on execution time aligns with common practices in evaluating non-functional requirements, especially for blockchain-based software platforms [61], [62], as functional requirements can vary significantly and are often implemented via heterogeneous protocol designs, making precise cross-method evaluations challenging.

TABLE III
Evaluation Setup (Adjusting N, D, L, M, T)

Parameter	N	D	L	M	T
N	10 to 100	10	10	1	10
D	10	10 to 100	10	1	10
L	10	10	10 to 100	1	10
M	10	10	10	1 to 10	10
T	10	100	10	1	10 to 100

To highlight the advantages of IBis, we compare it against two widely used architectural patterns: *Flat/Sequential Search* and *Full Graph Search*. These baselines represent common designs in real-world systems such as e-Commerce [62] and e-Healthcare [64], and also subsume recent blockchain-based copyright systems [65], [66], [67].

- *Flat/Sequential Search [62], [65], [66]:* Assets are treated as isolated records with no structured relationships. Systems of this type store either individual entries (e.g., CIDs and ownership metadata [65]) or simple one-way chains per asset [66]. Each query scans a linear list or full dataset to validate, retrieve, or compare records. For example, getDatasetLicense requires $O(|D|)$ time compared to $O(1)$ in IBis; getModelLicenses is $O(|D||M|)$ vs. $O(|M||T|)$, and getModelsByLicense is $O(|D||M|)$ vs. $O(|D| + |M||T|)$. These access costs grow rapidly with scale and offer no built-in mechanism for capturing cross-asset dependencies.
- *Full Graph Search [64], [67]:* More expressive systems represent licenses, evidence, and ownership as interconnected graph structures. For instance, [67] uses RDF triples and semantic links to connect works, claims, and complaints, with queries executed via external graph indexers. However, without pruning or indexing, operations such as getModelLicenses must walk all linked paths, resulting in $O(|M||T| + |D|)$ complexity compared to $O(|M||T|)$ in IBis. These systems capture richer semantics but suffer from high traversal costs and limited performance tuning.

IBis generalizes and improves upon both categories. It supports rich inter-asset semantics like the graph-based approaches while maintaining low-complexity queries through DAG indexing and caching. Because our design operates at the indexing layer, it can be added atop existing ledger infrastructures without changing their consensus or storage logic. This allows flat or graph-based systems to adopt IBis for efficient, provenance-aware queries and recursive license value flows.

Experimental parameters: In real-world scenarios, the framework hosts data, including dataset metadata, license, and model metadata, contributed by various AOs and COs. These stakeholders engage in executing functional operations (outlined in Table IV) to realize data provenance and copyright management. Ensuring the efficiency of operations, particularly those involving complex graph traversals, is paramount in this context. The experimental environment is set up the following parameters:

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on April 15,2026 at 09:46:28 UTC from IEEE Xplore. Restrictions apply.

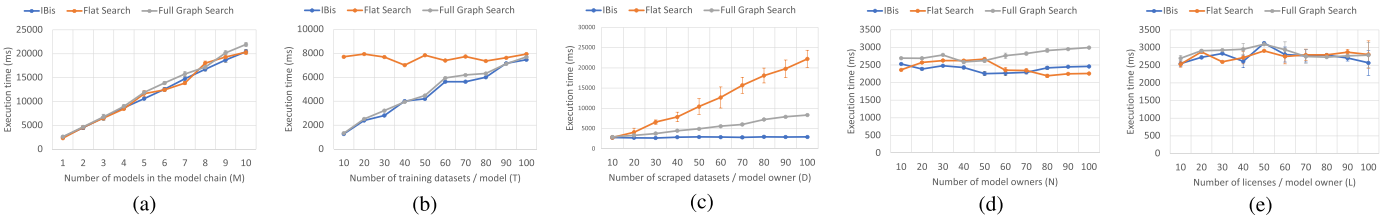


Fig. 8. Comparison between IBIS and baselines in terms of performance of fetching model licenses.

TABLE IV
OPERATIONS, COMPLEXITIES, AND AUTHORIZATIONS

Operation	Complexity	Authorisations
getDatasetLicense	$O(1)$	AOs
getModelLicenses	$O(\|M\|\|T\|)$	AOs
checkLicenseValidity	$O(\|K\|)$	AOs/COs
getLicensedDatasets	$O(\|D\|\|K\|)$	AOs
getDatasetsByLicense	$O(1)$	AOs
getModelsByLicense	$O(\|D\|+\|M\|\|T\|)$	AOs
getModelDatasets	$O(\|M\|\|T\|)$	AOs

- The framework accommodates N AOs, where each scrapes D datasets for model training. Therefore, the framework host a total of $N \times D$ datasets.
- Each AO acquired L licenses from various COs. Each dataset scraped by that AO is assigned one of the L licenses. For test purposes, the assignment is done randomly. Consequently, the total number of licenses hosted in the framework amounts to $N \times L$. In addition, some fraction of licenses may be associated with multiple datasets, while other licenses without datasets. This aligns well with real-world usage scenarios because AOs may collect licenses before scraping the corresponding dataset.
- To mirror the model retraining process in the real world, the experiment assumes each AO retrained a model $M-1$ times and obtained a chain of M models. Consequently, the total number of models hosted in the framework amounts to $N \times M$.
- Each model is trained on T datasets. For the testing purpose, those datasets are randomly picked from AO's D datasets.

There are five parameters during the experimental setup, namely N, D, L, M, and T. The system workload can be scaled up by increasing these parameters. In our evaluation, adhering to the control variates method, we measure the performance by varying each parameter individually while keeping the other parameters fixed. Table III lists the values of the four parameters that remain fixed while adjusting the remaining parameter.

Extreme cases: While Table III lists typical parameter ranges for evaluation, we intentionally do not explore extreme cases such as $M \gg T$, $M \ll T$, or very large D. These conditions are uncommon in practical AI workflows. In most scenarios, the average model retraining depth M ranges between 1 and 10, while the number of training datasets T remains within 10 to 100. To prevent unbounded graph growth and ensure runtime stability under rare edge cases, IBIS incorporates bounding strategies such as retraining checkpoints, anchor nodes, and lineage aggregation. These allow IBIS to maintain traversal efficiency and theoretical consistency even under potentially adversarial graph topologies. In all tested regimes, performance trends align with the expected complexities outlined in Table IV.

Complexity analysis: We provide the complexity analysis. Table IV lists the time complexity of operations and the entities authorized to perform them. We assume that the on-chain license registry, DMR, and MMR are implemented as hash maps on a smart contract, resulting in a time complexity of $O(1)$ for searching them.

Evaluation of fetching model licenses: Fig. 8 illustrates the variations in execution time for the getModelLicenses operation. Each data point represents the average execution time over 10 executions, with error bars indicating the standard deviation.

The execution time increases linearly with the number of models in the model chain (M) for all three methods, consistent with their theoretical complexities (cf. Fig. 8(a)). IBIS demonstrates the best performance due to its targeted graph traversal, which efficiently limits operations to T, the datasets directly linked to the models, resulting in a complexity of $O(MT)$. In contrast, Flat Search performs the worst, with a complexity of $O(MD)$ that involves scanning all D datasets for every model, causing significant overhead for large D. Full Graph Search performs better than Flat Search, benefiting from $O(MT)$ traversal for relevant datasets, but its additional $O(D)$ term for scanning all datasets for dataset-to-license mappings creates noticeable overhead, especially as D increases.

A similar linear trend appears as the number of training datasets per model (T) increases, with both IBIS and Full Graph Search scaling accordingly (cf. Fig. 8(b)). IBIS maintains efficiency by targeting only relevant datasets, while Full Graph Search incurs added cost due to the lack of direct dataset-to-license links. Flat Search remains constant, as its time is dominated by the total dataset count ($D = 100$), not the specific T linked to each model. Increasing the total number of scraped datasets per model owner (D) significantly impacts Flat Search, where execution time grows steeply due to its $O(MD)$ complexity (cf. Fig. 8(c)). Full Graph Search also slows down as it scans all datasets during license mapping. In contrast, IBIS remains stable by targeting only the T relevant datasets, demonstrating strong scalability for large-scale systems. Varying the number of model owners (N) and licenses per owner (L) has minimal impact on execution time across all methods (cf. Figs. 8(d)–8(e)). This aligns with theory, as N and L do not affect graph size or traversal. Efficient indexing (Section V-C) further ensures consistent query performance regardless of record count.

The outcomes also indicate that the values of the number of scraped datasets per model owner D, model owners N, and licenses per model owner L exert no discernible influence on the performance of getModelLicenses. In theory, these three parameters do not impact the graph size; hence they have negligible effect on performance. However, theoretically, they could affect performance as querying a record using its

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on April 15,2026 at 09:46:28 UTC from IEEE Xplore. Restrictions apply.

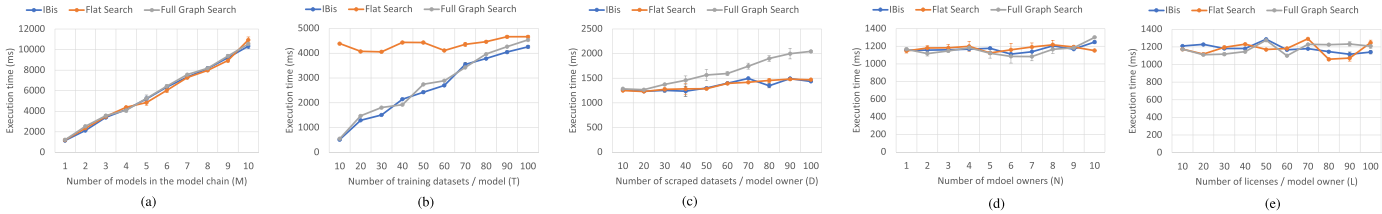


Fig. 9. Comparison between IBIS and baselines in terms of performance of fetching model datasets.

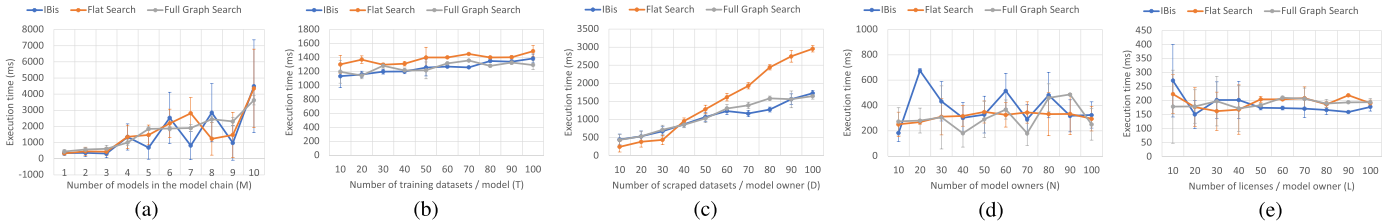


Fig. 10. Comparison between IBIS and baselines in terms of performance of fetching authorized models.

identifier might slow down with a greater number of records. However, our optimization efforts, such as designating data, model, and license identifiers as the primary key of a record (cf. Section V-C), mitigate any observable impact of increased record numbers. The results demonstrate that the performance of getModelLicenses operation remains consistent regardless of the number of model owners in the system, datasets they scrape, or licenses they acquire, thereby affirming the scalability of the operation.

Evaluation of fetching model datasets: Fig. 9 presents the execution time of the getModelDatasets operation under varying parameters. The results align with the theoretical complexities and highlight clear differences in performance as the parameters scale. As the number of models in the model chain (M) increases, the execution time grows linearly for all methods. Both IBIS and Full Graph Search demonstrate similar performance, consistent with their shared complexity of $O(MT)$, as they efficiently traverse only the T datasets associated with each model (cf. Fig. 9(a)). Flat Search, however, performs significantly worse, reflecting its $O(MD)$ complexity, which involves processing all D datasets for each model. This overhead becomes more pronounced as D increases.

Execution time also scales linearly with the number of training datasets per model (T) for IBIS and Full Graph Search, as their traversal depends on the number of datasets directly linked to the models (cf. Fig. 9(b)). Flat Search, by contrast, remains unaffected by changes in T, since its execution time is determined by D and M. This behavior underscores the inefficiency of Flat Search in targeting relevant datasets compared to the more focused approaches of IBIS and Full Graph Search.

When varying the number of scraped datasets per model owner (D), the number of model owners (N), and the number of licenses per model owner (L), the results reveal distinct trends (cf. Fig. 9(c)–(e)). Larger D impacts the performance of Full Graph Search, as its $O(MT + D)$ complexity includes scanning all datasets, leading to a steady increase in execution time. In contrast, IBIS maintains stable performance by limiting operations to T datasets, bypassing the irrelevant portions of D. Flat Search shows relatively constant execution times with increasing D, as it processes all D datasets regardless of their relevance, making the increase less noticeable. For N and L, the execution times remain relatively stable across all methods, as these parameters do not directly influence graph traversal. The consistent performance of IBIS in all cases validates its efficiency and scalability compared to Flat Search and Full Graph Search.

Evaluation of fetching authorized models: Fig. 10 illustrates the performance of the getModelsByLicense operation under varying system parameters. The execution time trends observed validate the theoretical complexities associated with each method. In Fig. 10(a), as the number of models in the model chain (M) increases, both IBIS and Full Graph Search exhibit a similar linear growth in execution time, consistent with their shared complexity of $O(MT + D)$. The efficient traversal of only the T datasets linked to the models, combined with a limited dependency on D, explains their comparable performance. Flat Search, on the other hand, incurs similar execution times in this experiment due to $T \simeq D$ in the experimental setup, aligning with its $O(MD)$ complexity. However, in scenarios where $T \ll D$, Flat Search would exhibit significantly higher execution times due to its inefficiency in processing all D datasets for each model.

As the number of training datasets per model (T) increases, the execution time grows linearly for both IBIS and Full Graph Search (cf. Fig. 10(b)). This behavior aligns with their $O(MT)$ complexity, as the traversal scales with the number of datasets connected to the models. In contrast, Flat Search remains unaffected by changes in T, as its performance depends solely on M and D (with D fixed at 100 in this setting). This distinction highlights the inefficiency of Flat Search in leveraging targeted traversal, which both IBIS and Full Graph Search effectively utilize to process only the relevant datasets.

When varying the number of scraped datasets per model owner (D) (cf. Fig. 10(c)), Flat Search shows a steep increase in execution time, consistent with its dependence on D in the $O(MD)$ complexity. Full Graph Search also exhibits growth with larger D, reflecting the added $O(D)$ term in its traversal. However, IBIS maintains relatively stable execution times, as its operations focus primarily on T datasets and avoid redundant processing of irrelevant datasets. This efficiency underscores the

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on April 15,2026 at 09:46:28 UTC from IEEE Xplore. Restrictions apply.

scalability of IBIS compared to the other methods when dealing with large-scale datasets.

Fig. 10(d)–(e) show the number of model owners (N) and licenses per model owner (L). The execution times remain nearly constant for all methods. These parameters do not directly affect the graph traversal operations required for fetching models by license. The stability of IBIS across these parameters further confirms its robustness and ability to handle increasing system size without compromising performance. Overall, the experimental results reaffirm the efficiency of IBIS, particularly as M, T, or D grows, while highlighting the limitations of Flat Search and the added overhead in Full Graph Search.

Discussions between evaluated operations: It is evident that the execution time of fetching model datasets using getModelDatasets is approximately half that of fetching model licenses getModelLicenses using. This phenomenon arises because fetching model datasets can be considered a sub-operation of fetching model licenses, which undertakes partial tasks compared to all. While fetching a license traverses the entire graph from a model to licenses, fetching a dataset terminates the traversal early at the dataset level. Moreover, because a training dataset consistently corresponds to a single license, traversing from datasets to licenses involves the same number of edges as traversing from models to datasets.

Moreover, the performance of fetching authorized model using getModelsByLicense displays distinct performance characteristics compared to the other two operations, particularly evidenced by its high standard deviations. This variance arises due to the different graph traversal directions. Additionally, the redundancy of datasets and licenses is only encountered in the traversal direction of the operation. Equivalently, while a dataset may not necessarily correspond to any model, a model invariably corresponds to some datasets. Similarly, while a license may not correspond to any datasets, a training dataset always corresponds to a license. Consequently, the graph traversal performance in the direction of getModelsByLicense exhibits greater statistical variability.

Overall, depending on the operation, the execution time can increase linearly with the number of scraped datasets per model owner D, training datasets per model T, or models in a model chain M. Meanwhile, the number of model owners N and licenses per model owner L do not significantly affect the execution time. This is consistent with our performance analysis in Table IV and validates scalability and feasibility.

B. Experiment-2: Lightweight Design

Experimental setting: We reuse the same Canton deployment: three t2.xlarge nodes (4 vCPUs, 16 GB RAM) sharing a docker-hosted PostgreSQL ledger store. To ensure consistency with Experiment-1, we fix the scenario to a single model-chain depth $M = 1$, ten training datasets ($T = 10$), ten scraped datasets ($D = 10$), one model owner ($N = 1$), and one active license ($L = 1$). This configuration is the minimal one that activates every contract path while matching the empirically observed 2.55 s latency for getModelLicenses. For alternative designs, we scale their results using relative overhead multipliers derived from micro-benchmarks, such that all designs execute the *identical* write–read cycle under the same ledger setup.

Flat/Sequential search: A baseline blockchain contract architecture where NFTs are minted and stored individually without any indexing or relationship metadata. Smart contracts execute basic CRUD operations but do not encode structural links between NFTs. Every access requires scanning all token metadata on-chain (e.g., via event logs or full contract state), leading to linear-time lookups. This mirrors naïve NFT deployments commonly seen on Ethereum or BNB Smart Chain [68] without metadata-aware extensions.

Full graph search: Contracts explicitly maintain NFT relationship structures as adjacency lists, stored directly in contract state (e.g., mapping from token IDs to downstream references). When resolving a license query, smart contracts must walk the graph edge-by-edge using iterative on-chain logic, incurring repeated read/write cycles across many keys. While this approach improves semantic expressiveness, it increases gas costs and latency due to repeated contract calls and deep on-chain state traversal.

zk-SNARK-based: A privacy-preserving approach implemented via cryptographic circuits embedded in smart contracts. Users submit zero-knowledge proofs (e.g., Groth16 [69] or PLONK [70] verifying ownership, compliance with licensing rules, or authorized derivation chains, without revealing sensitive data. On-chain verification is handled via precompiled contracts (e.g., on Ethereum), with each verification requiring thousands of elliptic curve operations. These proofs are appended to transaction payloads, increasing storage and consensus propagation time.

Homomorphic encryption (HE): Contracts store state variables in encrypted form (e.g., Paillier [71] or BFV ciphertexts [72]), and on-chain logic includes arithmetic over encrypted values. While preserving data confidentiality, every state update or evaluation (e.g., license price computation or access eligibility) requires encrypted computation routines and re-encryption steps. These must be committed on-chain along with ciphertext payloads, resulting in significant storage and consensus costs.

Secure multi-party computation (sMPC): Instead of computing licensing logic on-chain, smart contracts coordinate off-chain MPC among nodes. Each party holds a secret-shared state (e.g., Shamir shares [73]), and computation proceeds in rounds with on-chain commitments and verifications. The blockchain anchors session metadata, confirms rounds, and enforces outcomes. Frequent commitments and sync messages introduce significant network and consensus latency.

Overhead components: End-to-end latency is decomposed into six measurable components:

- *Metadata registration:* Serializing and submitting an NFT descriptor.
- *License Check:* Evaluating ACL terms in the contract state.
- *Proof Verification:* On-chain validation of cryptographic proofs (e.g., zk-SNARK, HE, and sMPC designs).
- *Graph Traversal:* Model → dataset → license edges.
- *Consensus Delay:* Time required by Canton's Global Synchronizer to finalize *each* write.
- *Storage I/O:* Persisting state changes to PostgreSQL.

Why consensus appears larger than the raw block period: Canton typically commits a single transaction in around 2 ms. However, our write–read cycle triggers *multiple* contract writes, including metadata insertion, license binding, reference update, and provenance checkpoint. The total of four commits yields 80 ms for cryptography-free designs and slightly more for cryptographic variants (due to larger payloads). These cumulative delays appear as 80–120 ms in Fig. 11, and align with the 2.55 s measured for IBIS in our benchmark.

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on April 15,2026 at 09:46:28 UTC from IEEE Xplore. Restrictions apply.

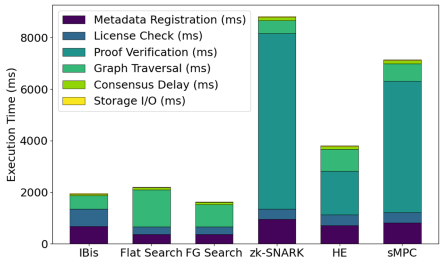


Fig. 11. Scaled execution time per overhead component (ms).

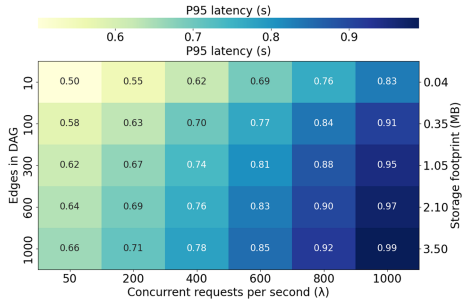


Fig. 12. Scalability visualization across different edges, on-chain storage, concurrent requests per second, and the 95-th percentile delay.

Discussion: Our evaluation reveals clear performance advantages of IBIS over other blockchain-based methods.

(1) Indexing instead of brute-force or cryptographic proofs: IBIS completes the write–read cycle in 2.55 s, as its only variable cost is the indexed DAG traversal ($\sim$0.5 s for M=1, T=10); the remaining overhead—four Canton commits and PostgreSQL flushes—totals just $\sim$0.08 s. Flat and Full Graph Search avoid cryptographic costs but rely on exhaustive graph traversal: a linear scan takes 1.44 s in Flat Search, and an edge-by-edge walk takes 0.86 s in Full Graph Search, resulting in total latencies of 2.2 s and 1.6 s, respectively. As M or T grows, the cost in IBIS increases linearly, while for the baselines it grows quadratically.

(2) Proof-centric designs incur irreducible latency: Crypto-heavy designs substitute graph traversal with verification overhead. A zk-SNARK verifier executes around 3,400 elliptic curve pairings ($\sim$ 2 ms each), yielding 6.8 s of proof checking and further bloating consensus payloads, leading to a total delay exceeding 8.8 s. HE-based models incur 1.7 s for ciphertext computation, totaling 3.8 s. sMPC setups, with multiple communication rounds and quorum-based computation, reach 7.1 s. These designs suffer from per-call proof overheads that cannot be amortized or cached, in contrast to IBIS, which uses lightweight multi-party signatures and Canton's sub-20ms commit path to preserve both auditability and scalability.

C. Experiment-3: Scalability

Experimental setting: Using the same Canton deployment used in previous experiments, we sweep two orthogonal stress axes: (i) the number of directed edges preloaded in the on-chain DAG ($|E| \in \{10, 100, 300, 600, 1000\}$), and (ii) the incoming request rate ($\lambda \in \{50, 200, 400, 600, 800, 1000\}$ req/s). For each $(|E|, \lambda)$ pair we record the **P95 endtoend latency**, i.e. the 95th percentile delay from the client's first metadata write to receipt of the final getModelLicenses response. We visualize the results in Fig. 12: rows are DAG sizes/storage footprint, columns are request rates, and cell color is *P95 latency*.

Metrics captured: Concurrent requests λ reflect horizontal load; they are the principal driver of throughput saturation. *Edges* $|E|$ (or, equivalently, the *storage footprint*) captures vertical scale: how large the reference network can grow before query performance degrades. *P95 latency* is chosen because tail delay, not the mean, determines userperceived responsiveness in a multitenant marketplace.

Why P95 Latency: P95 latency in our scalability tests captures the 95th-percentile end-to-end delay of a complete, honest write–read cycle, beginning with NFT metadata submission and ending with the getModelLicenses response. It accounts for all key steps, including metadata registration, license binding, reference updates, provenance tracking, DAG lookups, PostgreSQL I/O, and gRPC communication. Unlike the average execution time reported in Experiment (1) (e.g., 2.55 s for getModelLicenses under single-threaded conditions), P95 latency captures tail performance under real-world concurrency with up to 1000 requests per second. Under light load and small DAGs (e.g., $|E|$=10, λ=50), latency remains around 0.55 s, closely matching the DAG portion of prior benchmarks. As load and DAG size grow, Fig. 12 shows how latency evolves and how much margin remains before breaching service expectations. This metric confirms that IBIS maintains sub-second tail latency for $|E| \leq 10^3$ and $\lambda \leq 800$ req/s, and stays within 1.2 s even at peak load, supporting interactive use at scale.

Results and insights: Fig. 12 shows that for the smallest DAG ($|E|$=10), *P95 latency* is 0.48 s at 50 req/s and rises modestly to 0.83 s at 1000 req/s, with the Canton commit path being the dominant factor in this regime rather than DAG lookups. As $|E|$ increases, latency grows sub-logarithmically: the worst-case point at $|E|$=1000, λ=1000 reaches only 1.17 s—about 2.4$\times$ the baseline despite a 100$\times$ expansion in state size. This aligns with the expected $\mathcal{O}(\log |E|)$ cost of indexed traversal. From the view of on-chain storage, even with 3.5 kB $\times$ 1000 $\approx$ 3.5 MB of added state, the system remains within the 1.2 s interactive response threshold. Since storage grows linearly with $|E|$ but latency grows only logarithmically, the heat map cleanly decouples *data volume* from *performance impact*.

D. Experiment-4: Security and Robustness

Experimental setting: This experiment builds upon the same Canton-based deployment and reuses the scalability harness to assess security-critical behavior under adversarial conditions. We simulate a mix of honest and malicious client requests, keeping validator nodes honest to isolate application-layer resilience. Three workload regimes are tested: *Baseline* (0% malicious), *Under Attack* (20% malicious), and *Heavy Attack* (up to 50% malicious). Each request passes through the same metadata registration and verification path used in previous experiments, allowing us to measure not only throughput and tail latency but the effectiveness of embedded security checks.

Attack vectors: Two attack types are considered: *provenance spoofing (P)* and *signature tampering (S)*. Provenance spoofing mimics the plagiarism of datasets or models, where the attacker copies an existing NFT, introduces minimal modifications, and submits it as a new asset. These are caught using content-addressable hashes (CIDs) and optional perceptual similarity checks. Signature tampering includes malformed multi-party ECDSA signatures or stale signature replays using

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on April 15,2026 at 09:46:28 UTC from IEEE Xplore. Restrictions apply.

TABLE V
EVALUATION OF THE ROBUSTNESS OF IBIS UNDER PLAGIARISM AND SIGNATURE TAMPER

Scenario	DAG Edges	Honest req/s	Malicious %	Attack Mix	Performance			Attack Mitigation		
					Throughput (tx/s)	P95 Latency (s)	Storage (MB)	Plag. Reject %	Sig. Reject %	Overall Reject %
Baseline	100	100	0	None	99	0.55	0.35	-	-	-
	500	500	0	None	496	0.82	1.75	-	-	-
	900	750	0	None	743	0.91	3.15	-	-	-
Under Attack	100	80	20	$P+S$	79	0.67	0.35	87%	92%	90%
	500	400	20	P	396	0.85	1.75	96%	-	96%
	900	600	20	S	593	1.16	3.15	-	99%	99%
Heavy Attack	100	70	30	$P+S$	68	0.81	0.35	90%	97%	94%
	500	350	40	$P+S$	337	1.12	1.75	88%	96%	93%
	900	450	50	$P+S$	425	1.25	3.15	91%	98%	95%

- P = provenance (plagiarism) spoof; S = signature tamper.
- Throughput is the number of *accepted* transactions per second after rejecting malicious ones.
- Storage column represents estimated per-node state size at the displayed DAG-edge count.
- Attack-mitigation columns report the percentage of malicious requests successfully rejected. "Plag. Reject %" quantifies the rate at which spoofed provenance attempts are blocked. "Sig. Reject %" measures the rate of invalid or replayed signatures being rejected. "Overall Reject %" is the aggregate success rate for all injected malicious traffic.
- Dashes ("–") indicate the metric is not applicable under the given attack mix, such as when no relevant attack vector is present in the experiment.
- Malicious % denotes the fraction of incoming client requests that are adversarial; all validator (consensus) nodes are assumed honest, so ledger safety is never compromised.

previously valid nonces or timestamps. These are intercepted through nonce-based replay protection and on-chain verification of ECDSA integrity.

Consensus integrity: Despite up to 50% of client traffic being malicious in the *Heavy Attack* group, ledger safety remains intact. These adversarial requests are dropped at the node level and never reach the consensus layer. Canton's Global Synchronizer protocol [] continues to finalize only valid transactions,and the underlying ordering service is unaffected apart from the additional CPU cycles spent rejecting forged submissions.

Results and insights: Table V shows that under normal operation (*Baseline*), IBIS maintains sub-second *P95 latency* (0.55–0.91 s) and nearly full throughput (up to 743 tx/s). In the *Under Attack* scenarios, throughput degradation is minimal (e.g., from 750 to 593 tx/s), while rejection rates[4] remain high: 96% for provenance, 99% for signature tampering. Even under *heavy attack* conditions with 50% adversarial input, throughput remains above 425 tx/s, and the rejection engine still filters 93–95% of valid requests, with *P95 latency* staying below 1.3 s. These results demonstrate that IBIS preserves both performance and integrity under adversarial stress, offering robust protections against critical threats while sustaining interactive responsiveness.

VII. CONCLUSION

This paper introduced IBIS, a blockchain-based system for managing data provenance, lineage, and copyright in AI workflows. IBIS enforced license constraints over model retraining, supported multi-party agreement signing, and implemented access control for confidentiality. Built on the Daml ledger and Canton blockchain, IBIS demonstrated strong scalability across diverse workloads. Future work includes optimizing on-chain graph structures and extending coverage to other AI lifecycle stages such as data cleaning, testing, and explanation.

REFERENCES

[1] R. Bommasani et al., "On the opportunities and risks of foundation models," 2021, *arXiv:2108.07258*.
[2] W. X. Zhao et al., "A survey of large language models," 2023, *arXiv:2303.18223*.
[3] Y. Zhu et al., "Large language models for information retrieval: A survey," 2023, *arXiv:2308.07107*.

[4]Rejection rates are conservatively capped below 100% to reflect practical realities such as threshold-based similarity detection and rare clock skew during nonce validation—factors that mirror real-world deployment conditions without compromising system reliability.

[4] L. Bonifacio, H. Abonizio, M. Fadaee, and R. Nogueira, "Inpars: Data augmentation for information retrieval using large language models," 2022, *arXiv:2202.05144*.
[5] J. Li, T. Tang, W. X. Zhao, J.-Y. Nie, and J.-R. Wen, "Pretrained language models for text generation: A survey," 2022, *arXiv:2201.05273*.
[6] J. Chen, H. Lin, X. Han, and L. Sun, "Benchmarking large language models in retrieval-augmented generation," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 17 754–17 762.
[7] N. Carlini et al., "Extracting training data from large language models," in *Proc. USENIX Secur. Symp.*, 2021, pp. 2633–2650.
[8] J. Hoffmann, "An empirical analysis of compute-optimal large language model training," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 30016–30030.
[9] T. Chu, Z. Song, and C. Yang, "How to protect copyright data in optimization of large language models," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 17 871–17 879.
[10] N. Vyas, S. M. Kakade, and B. Barak, "On provable copyright protection for generative models," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2023, pp. 35 277–35 299.
[11] Z. Yu, Y. Wu, N. Zhang, C. Wang, Y. Vorobeychik, and C. Xiao, "CodeiPPrompt: Intellectual property infringement assessment of code language models," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2023, pp. 40 373–40 389.
[12] Q. Lu, Y. Luo, L. Zhu, M. Tang, X. Xu, and J. Whittle, "Developing responsible chatbots for financial services: A pattern-oriented responsible AI engineering approach," *IEEE Intell. Syst.*, vol. 38, no. 6, pp. 42–51, Nov./Dec. 2023.
[13] Q. Lu, L. Zhu, X. Xu, J. Whittle, D. Zowghi, and A. Jacquet, "Operationalizing responsible AI at scale: CSIRO data61's pattern-oriented responsible AI engineering approach," *Commun. ACM*, vol. 66, no. 7, pp. 64–66, 2023.
[14] A. Power, "Licensing agreements," *Miss. LJ*, vol. 42, 1970, Art. no. 169.
[15] M. Benjamin, P. Gagnon, N. Rostamzadeh, C. Pal, Y. Bengio, and A. Shee, "Towards standardization of data licenses: The montreal data license," 2019, *arXiv: 1903.12262*.
[16] D. Contractor et al., "Behavioral use licensing for responsible AI," in *Proc. ACM Conf. Fairness Accountability Transparency*, 2022, pp. 778–788.
[17] D. Siddarth et al., "How AI fails us," 2021, *arXiv:2201.04200*.
[18] R. Li et al., "How do smart contracts benefit security protocols," 2022, *arXiv:2202.08699*.
[19] T. L. Nguyen et al., "Blockchain-empowered trustworthy data sharing: Fundamentals, applications, and challenges," *ACM Comput. Surv.*, vol. 57, no. 8, pp. 1–36, 2025.
[20] X. Xu, I. Weber, and M. Staples, *Architecture for Blockchain Applications*. Berlin, Germany: Springer, 2019.
[21] W. Zhang et al., "Blockchain-based distributed compliance in multinational corporations' cross-border intercompany transactions," in *Proc. Adv. Inf. Commun. Netw.*, K. Arai, S. Kapoor, and R. Bhatia, Eds. Cham, Springer Int. Publishing, 2019, pp. 304–320.
[22] T. Scott, A. L. Post, J. Quick, and S. Rafiqi, "Evaluating feasibility of blockchain application for DSCSA compliance," *SMU Data Sci. Rev.*, vol. 1, no. 2, pp. 1–26, 2018.
[23] M. Allena, "Blockchain technology and regulatory compliance: Towards a cooperative supervisory model," *Eur. Rev. Digit. Admin. Law*, vol. 2, pp. 37–43, 2022.
[24] O. Ural and K. Yoshigoe, "Survey on blockchain-enhanced machine learning," *IEEE Access*, vol. 11, pp. 145 331–145 362, 2023.

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on April 15,2026 at 09:46:28 UTC from IEEE Xplore. Restrictions apply.

[25] A. A. Hussain and F. Al-Turjman, "Artificial intelligence and blockchain: A review," *Trans. Emerg. Telecommun. Technol.*, vol. 32, no. 9, 2021, Art. no. e4268.

[26] Y. Liu, F. R. Yu, X. Li, H. Ji, and V. C. Leung, "Blockchain and machine learning for communications and networking systems," *IEEE Commun. Surv. Tut.*, vol. 22, no. 2, pp. 1392–1431, Second Quarter 2020.

[27] A. Kiayias and P. Lazos, "SoK: Blockchain governance," in *Proc. ACM Conf. Adv. Financial Technol.*, 2022, pp. 61–73.

[28] Y. Liu, Q. Lu, L. Zhu, H.-Y. Paik, and M. Staples, "A systematic literature review on blockchain governance," *J. Syst. Softw.*, vol. 197, 2023, Art. no. 111576.

[29] X. Xiong, M. Huth, and W. Knottenbelt, "REGKYC: Supporting privacy and compliance enforcement for KYC in blockchains," in *Proc. IEEE Int. Conf. Blockchain Cryptocurrency*, 2025, pp. 1–10.

[30] H. Jia et al., "Proof-of-learning: Definitions and practice," in *Proc. IEEE Symp. Secur. Privacy*, 2021, pp. 1039–1056.

[31] Y. Lan, Y. Liu, B. Li, and C. Miao, "Proof of learning (PoLe): Empowering machine learning with consensus building on blockchains," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 16 063–16 066.

[32] Z. Wang, R. Sun, E. Lui, T. Zhou, Y. Wen, and J. Sun, "AIArena: A blockchain-based decentralized ai training platform," in *Proc. Web Conf.*, 2024, pp. 1375–1379.

[33] Q. Wang, G. Yu, T. Bi, H. D. Bandara, and S. Chen, "A responsibility pillar: Exploring and assessing decentralized intelligence," in *Proc. IEEE Int. Conf. Blockchain*, 2024, pp. 412–421.

[34] N. B. Somy et al., "Ownership preserving AI market places using blockchain," in *Proc. IEEE Int. Conf. Blockchain*, 2019, pp. 156–165.

[35] G. Yu et al., "Maximizing NFT incentives: References make you rich," 2024, *arXiv:2402.06459*.

[36] Z. Wang et al., "SoK: Decentralized AI (DeAI)," 2024, *arXiv:2411.17461*.

[37] Z. Chen, "Catch me if you can: Detecting unauthorized data use in training deep learning models," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2024, pp. 5098–5100.

[38] A. Bernauer et al., "Daml: A smart contract language for securely automating real-world multi-party business workflows," 2023, *arXiv:2303.03749*.

[39] D. A. C. Team, "Canton: A Daml based ledger interoperability protocols," Digital Asset, Tech. Rep., 2020. [Online]. Available: https://www.digitalasset.com/hubfs/Canton/canton-whitepaper.pdf?hsLang=en

[40] D. Kreuzberger, N. Kühl, and S. Hirschl, "Machine learning operations (MLOps): Overview, definition, and architecture," *IEEE Access*, vol. 11, pp. 31 866–31 879, 2023.

[41] J. Litman, "What notice did," *Boston Univ. Law Rev.*, vol. 96, pp. 717–744, 2016.

[42] Q. Wang, R. Li, Q. Wang, and S. Chen, "Non-fungible token (NFT): Overview, evaluation, opportunities and challenges," 2021, *arXiv:2105.07447*.

[43] A. Savelyev, "Copyright in the blockchain era: Promises and challenges," *Comput. Law Secur. Rev.*, vol. 34, no. 3, pp. 550–561, 2018.

[44] N. Jing, Q. Liu, and V. Sugumaran, "A blockchain-based code copyright management system," *Inf. Process. Manage.*, vol. 58, no. 3, 2021, Art. no. 102518.

[45] B. Wang, S. Jiawei, W. Wang, and P. Zhao, "Image copyright protection based on blockchain and zero-watermark," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 4, pp. 2188–2199, Jul./Aug. 2022.

[46] A. Borzunov et al., "Distributed inference and fine-tuning of large language models over the internet," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 12312–12331.

[47] Y. Gao, Z. Song, and J. Yin, "GradientCoin: A peer-to-peer decentralized large language models," 2023, *arXiv.2308.10502*.

[48] X. Liu et al., "Decentralized federated unlearning on blockchain," 2024, *arXiv:2402.16294*.

[49] G. Yu et al., "IronForge: An open, secure, fair, decentralized federated learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 1, pp. 354–368, Jan. 2025.

[50] C. Meurisch and M. Mühlhäuser, "Data protection in AI services: A survey," *ACM Comput. Surv.*, vol. 54, pp. 1–38, 2021.

[51] B. Gedik and L. Liu, "Protecting location privacy with personalized k-anonymity: Architecture and algorithms," *IEEE Trans. Mobile Comput.*, vol. 7, no. 1, pp. 1–18, Jan. 2008.

[52] D. Xu, S. Yuan, and X. Wu, "Achieving differential privacy and fairness in logistic regression," in *Proc. World Wide Web Conf.*, 2019, pp. 594–599.

[53] J. Zhang et al., "Protecting intellectual property of deep neural networks with watermarking," in *Proc. Asia Conf. Comput. Commun. Secur.*, 2018, pp. 159–172.

[54] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2016, pp. 201–210.

[55] B. D. Rouhani, M. S. Riazi, and F. Koushanfar, "Deepsecure: Scalable provably-secure deep learning," in *Proc. Annu. Des. Automat. Conf.*, 2018, pp. 1–6.

[56] R. Shokri and V. Shmatikov, "Privacy-preserving deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2015, pp. 1310–1321.

[57] S. Servia-Rodríguez, L. Wang, J. R. Zhao, R. Mortier, and H. Haddadi, "Privacy-preserving personal model training," in *Proc. IEEE/ACM 3rd Int. Conf. Internet-of-Things Des. Implementation*, 2018, pp. 153–164.

[58] W. Liang, D. Zhang, X. Lei, M. Tang, K.-C. Li, and A. Y. Zomaya, "Circuit copyright blockchain: Blockchain-based homomorphic encryption for IP circuit protection," *IEEE Trans. Emerg. Top. Comput.*, vol. 9, no. 3, pp. 1410–1420, Third Quarter 2021.

[59] Y. Liu et al., "Blockchain-empowered lifecycle management for AI-generated content products in edge networks," *IEEE Wirel. Commun.*, vol. 31, no. 3, pp. 286–294, Jun. 2024.

[60] E. Androulaki et al., "Hyperledger fabric: A distributed operating system for permissioned blockchains," in *Proc. EuroSys Conf.*, 2018, pp. 1–15.

[61] Q. Lu, X. Xu, Y. Liu, I. Weber, L. Zhu, and W. Zhang, "uBaaS: A unified blockchain as a service platform," *Future Gener. Comput. Syst.*, vol. 101, pp. 564–575, 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167739X18319873

[62] Z. Guan, N. Wang, X. Fan, X. Liu, L. Wu, and S. Wan, "Achieving secure search over encrypted data for E-commerce: A blockchain approach," *ACM Trans. Internet Technol.*, vol. 21, no. 1, pp. 1–17, Dec. 2020. [Online]. Available: https://doi.org/10.1145/3408309

[63] M. Zhaofeng, W. Xiaochang, D. K. Jain, H. Khan, G. Hongmin, and W. Zhen, "A blockchain-based trusted data management scheme in edge computing," *IEEE Trans. Ind. Informat.*, vol. 16, no. 3, pp. 2013–2021, Mar. 2020.

[64] S. Delaney, D. Schmidt, and C. C. K. Chan, "Present clinicians with the most relevant patient healthcare data through the integration of graph DB, Semantic Web and blockchain technologies," in *Proc. Adv. Inf. Netw. Appl.*, L. Barolli, I. Woungang, and T. Enokido, Eds. Cham: Springer International Publishing, 2021, pp. 606–616.

[65] M. M. Islam and H. P. In, "Decentralized global copyright system based on consortium blockchain with proof of authority," *IEEE Access*, vol. 11, pp. 43 101–43 115, 2023.

[66] X. Chen, A. Yang, J. Weng, Y. Tong, C. Huang, and T. Li, "A blockchain-based copyright protection scheme with proactive defense," *IEEE Trans. Serv. Comput.*, vol. 16, no. 4, pp. 2316–2329, Jul./Aug. 2023.

[67] R. García, A. Cediel, M. Teixidó, and R. Gil, "Semantics and non-fungible tokens for copyright management on the metaverse and beyond," *ACM Trans. Multimedia Comput. Commun. Appl.*, vol. 20, no. 7, pp. 1–20, Mar. 2024. [Online]. Available: https://doi.org/10.1145/3585387

[68] F. Cernera, M. L. Morgia, A. Mei, and F. Sassi, "Token spammers, rug pulls, and sniper bots: An analysis of the ecosystem of tokens in ethereum and in the binance smart chain (BNB)," in *Proc. USENIX Secur. Symp.*, Anaheim, CA, USENIX Association, 2023, pp. 3349–3366. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/cernera

[69] C. E. Quintero-Narvaez and R. Monroy-Borja, "Zero-knowledge proofs for questionnaire result verification in smart contracts," in *Proc. Mex. Int. Conf. Comput. Sci.*, 2023, pp. 1–5.

[70] D. Čapko, S. Vukmirović, and N. Nedić, "State of the art of zero-knowledge proofs in blockchain," in *Proc. 30th Telecommun. Forum*, 2022, pp. 1–4.

[71] M. Ghadamyari and S. Samet, "Privacy-preserving statistical analysis of health data using paillier homomorphic encryption and permissioned blockchain," in *Proc. IEEE Int. Conf. Big Data*, 2019, pp. 5474–5479.

[72] P.-C. Chen, T.-H. Kuo, and J.-L. Wu, "A study of the applicability of ideal lattice-based fully homomorphic encryption scheme to ethereum blockchain," *IEEE Syst. J.*, vol. 15, no. 2, pp. 1528–1539, Jun. 2021.

[73] Y. Zhu, X. Song, S. Yang, Y. Qin, and Q. Zhou, "Secure smart contract system built on SMPC over blockchain," in *Proc. IEEE Int. Conf. Internet Things-IEEE Green Comput. Commun.-IEEE Cyber Phys. Social Comput.-IEEE Smart Data*, 2018, pp. 1539–1544.

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on April 15,2026 at 09:46:28 UTC from IEEE Xplore. Restrictions apply.



Qin Wang (Member, IEEE) received the bachelor's degree from Northwestern Polytechnical University, Xi'an, China, in 2015, the master's degree from Beihang University, Beijing, China, in 2018, and the PhD degree from the Swinburne University of Technology, Melbourne, Australia, in 2022. He is a senior research scientist with CSIRO Data61 and an adjunct lecturer with UNSW, Sydney, Australia. His expertise spans emerging Web3 techniques as well as the fundamental aspects of consensus protocols, such as scalability, security, and privacy.



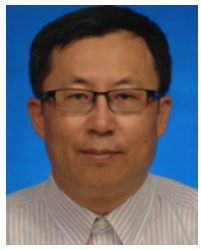
Guangsheng Yu (Member, IEEE) received the BSc and MSc degrees from the University of New South Wales, Sydney, Australia, from 2011 to 2015 and the PhD degree with the Faculty of Engineering and Information Technology, UTS, Australia. He is currently a lecturer with the University of Technology Sydney (UTS), Australia. Previously, he was a postdoctoral research fellow with CSIRO Data61, Australia. His main research interests lie in blockchain and federated learning.



Yilin Sai (Student Member, IEEE) received the BEng(hons) degree from the University of Sydney. He is a senior software engineer with CSIRO Data61 and currently working toward the PhD degree with the University of New South Wales, Sydney, Australia. With expertise in full-stack development, he specializes in building and delivering innovative, robust, and scalable cloud software platforms and solutions. His research interests include blockchain, distributed computing, Big Data analysis, Internet of Things, deep learning, and network security.



H. M. N. Dilum Bandara (Member, IEEE) received the MS and PhD degrees from Colorado State University, USA. He is a senior research scientist with CSIRO Data61, Australia. He is also a conjoint senior lecturer with UNSW, Sydney. His research interests include distributed systems (e.g., blockchain, cloud, P2P), computer security, software architecture, as well as multidisciplinary applications of those technologies. He is a charted engineer with IESL.



Shiping Chen (Senior Member, IEEE) is a principal research scientist with CSIRO Data61. He also holds a conjunct professor with UNSW Sydney. He is actively involved in computing research communities through publications, journal editorships and conference PC services, including WWW, EDOC, ICSOC, and IEEE ICWS/SCC/CLOUD. His research interests include secure data storage & sharing and secure multi-party collaboration. He is an IET fellow.

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on April 15,2026 at 09:46:28 UTC from IEEE Xplore. Restrictions apply.